

## Atelier TP — Soirée 18

### *Persistance — sérialisation JSON et reconstruction polymorphe*

Cet atelier capitalise sur le constructeur alternatif vu en S13 ( `depuis_chaine_csv` , ici en simple rappel) et sur les collections d'objets de la S17. Il introduit la **persistance JSON** : sauvegarder un catalogue dans un fichier, puis le recharger à l'identique. Travaillez individuellement.

**Déroulé du rendu (dépôt GitHub).** Forkez le dépôt du cours, puis ouvrez votre fork dans VS Code (les commandes terminal équivalentes figurent dans le [README.md](#)). Vous complétez les fichiers squelette **en place** : votre fork *est* votre copie de travail, le dépôt du cours n'est jamais modifié. Faites **un commit par exercice**. Vos réponses écrites (toutes les questions AA3 et le diagnostic) vont dans [reponses.txt](#). En fin de séance, ouvrez une **Pull Request** vers le dépôt du cours : elle est le point de rendu et l'espace de retour ; elle ne sera pas fusionnée.

## Contenu du dépôt

Votre fork contient les fichiers suivants. Vous complétez les squelettes et n'avez pas à créer de nouveau fichier :

- [README.md](#) — consignes d'installation et de rendu (panneau VS Code et équivalents terminal).
- [livre\\_s18\\_squelette.py](#) — les classes `Livre` , `LivreNumerique` et `LivreAudio` . Tout le comportement de la S14 vous est fourni *complet* ; seules les deux méthodes de sérialisation JSON ( `to_dict` et `from_dict` ) sont à écrire. Elles lèvent pour l'instant `NotImplementedError` .
- [catalogue\\_persistance\\_squelette.py](#) — les fonctions d'entrée/sortie JSON du catalogue et le registre de reconstruction, à écrire (exercices 3 et 4).
- [chargement\\_buggy.py](#) — un fichier de chargement délibérément défectueux, support du diagnostic de l'exercice 6.
- [reponses.txt](#) — gabarit pour vos réponses écrites.

Ne renommez pas les fichiers squelette : `catalogue_persistance_squelette.py` et `chargement_buggy.py` importent depuis `livre_s18_squelette`.

## 1. Exercice 1 — Sérialiser un livre en JSON (AA1)

### Énoncé

Vous implémentez la sérialisation JSON pour la classe `Livre` seule (les sous-classes viennent à l'exercice 2).

1. Implémentez `to_dict(self)`. Elle retourne un dictionnaire contenant : un champ `"type"` valant la chaîne `"Livre"`, puis `titre`, `auteur`, `isbn`, `nb_pages`, `annee` et `disponible`. Toutes les valeurs doivent être de types simples, directement sérialisables par `json`.
2. Implémentez la `@classmethod` `from_dict(cls, donnees)`. Deux exigences :
  - la reconstruction passe par le constructeur (`cls(...)`), pas par une écriture directe des attributs ;
  - l'état est restauré : si `donnees["disponible"]` est faux, le livre reconstruit doit être emprunté. Vous obtenez cet état **uniquement** via la méthode publique `emprunter()`, jamais en écrivant l'attribut privé.
3. Vérifiez l'aller-retour JSON à la console, en passant par une chaîne :

```
>>> import json
>>> d = livre.to_dict()
>>> Livre.from_dict(json.loads(json.dumps(d))) == livre
True
```

- ✓ `to_dict` produit un dictionnaire marqué `"type": "Livre"` avec tous les champs attendus.
- ✓ `from_dict` reconstruit via `cls(...)` et restaure la disponibilité par `emprunter()`.
- ✓ L'aller-retour redonne un livre égal à l'original (égalité par ISBN).

## 2. Exercice 2 — Enrichir la sérialisation dans les sous-classes (AA1)

### Énoncé

Vous étendez `to_dict` et `from_dict` aux deux sous-classes, **par enrichissement** et non par réécriture.

1. Dans `LivreNumerique`, redéfinissez `to_dict` : appelez `super().to_dict()`, corrigez le champ `"type"` en `"LivreNumerique"`, et ajoutez le champ `"format_fichier"`. Procédez de même dans `LivreAudio` avec `"duree_minutes"`.
2. Dans chaque sous-classe, redéfinissez `from_dict` pour lire son champ propre et appeler son constructeur avec le bon nombre d'arguments. C'est la même nécessité qu'en S13 pour `depuis_chaine_csv` : le constructeur attend un argument de plus.
3. Vérifiez à la console :

```
>>> LivreNumerique("Fondation", "Asimov",
...               "9782070360536", 256, 1951, "EPUB").to_dict()["type"]
'LivreNumerique'
```

4. **Question AA3** (dans `reponses.txt`). Pourquoi `to_dict` appelle-t-il `super().to_dict()` (enrichissement), alors que `taille_estimee()` de la S14 ne faisait *pas* appel à `super()` (remplacement) ? Qu'est-ce qui distingue les deux situations ?

- ✓ `to_dict` des sous-classes appelle `super().to_dict()` et ajoute le seul champ propre.
- ✓ Le champ `"type"` reflète la classe concrète dans les trois cas.
- ✓ `from_dict` de chaque sous-classe lit son champ propre et appelle le bon constructeur.
- ✓ La réponse AA3 oppose correctement enrichissement et remplacement.

### 3. Exercice 3 — Sauvegarder et recharger un catalogue (AA1)

#### Énoncé

C'est l'exercice central. Vous écrivez la persistance JSON d'un catalogue *hétérogène* et sa relecture, en restaurant le type exact de chaque livre.

1. Implémentez la fabrique `livre_depuis_dict(donnees)`. Elle lit le champ `"type"`, choisit la classe correspondante, et délègue à la `from_dict` de cette classe. Si le champ `"type"` est absent ou inconnu, levez une `ValueError` explicite. *À ce stade, une cascade de `if` / `elif` est acceptable ; vous l'améliorerez à l'exercice 4.*
2. Implémentez `sauvegarder_catalogue_json(livres, chemin)`. Elle transforme chaque livre par `to_dict` (dispatch polymorphe, aucun `isinstance`), puis écrit la liste avec `json.dump`. Ouvrez le fichier en écriture, en `encoding="utf-8"`.
3. Implémentez `charger_catalogue_json(chemin)`. Elle relit avec `json.load`, puis reconstruit chaque livre via `livre_depuis_dict`.
4. Démontrez l'aller-retour sur un catalogue contenant au moins un `Livre`, un `LivreNumerique` et un `LivreAudio`, dont au moins un est emprunté avant la sauvegarde. Vérifiez que chaque livre retrouve son type exact, son champ propre et son état.

La transformation objet  $\rightarrow$  dictionnaire est portée par les *classes* (`to_dict`). L'ouverture du fichier et l'appel à `json.dump` sont portés par les *fonctions* de `catalogue_persistance_squelette.py`. Le module des classes n'importe pas `json`.

- ✓ `livre_depuis_dict` restaure le type exact et lève `ValueError` sur un type absent ou inconnu.
- ✓ `sauvegarder_catalogue_json` n'utilise aucun `isinstance`.
- ✓ L'aller-retour préserve type concret, champ propre et disponibilité.
- ✓ Le fichier produit est du JSON valide et lisible (indenté).

## 4. Exercice 4 — Le registre de reconstruction (AA3)

### Énoncé

Vous remplacez la cascade `if` / `elif` de la fabrique par un **registre**.

1. Complétez le dictionnaire `_FABRIQUES` de sorte qu'il associe chaque discriminateur (`"Livre"`, `"LivreNumerique"`, `"LivreAudio"`) à la classe correspondante.
2. Réécrivez `livre_depuis_dict` pour qu'elle cherche la classe dans `_FABRIQUES` et l'invoque, sans `if` / `elif` sur le type. Le type inconnu (clé absente) doit toujours lever une `ValueError` explicite.
3. **Question AA3** (dans `reponses.txt`). Énoncez le **principe ouvert/fermé**. Expliquez en quoi la cascade le violait, et en quoi le registre le respecte. Décrivez ce qu'il faudrait modifier, dans chaque version, pour ajouter demain un type `LivreBraille`.

- ✓ `_FABRIQUES` associe correctement les trois discriminateurs aux trois classes.
- ✓ `livre_depuis_dict` n'utilise plus de `if` / `elif` et conserve la `ValueError` sur type inconnu.
- ✓ La réponse AA3 énonce le principe ouvert/fermé et l'applique aux deux versions.

## 5. Exercice 5 — JSON ou pickle ? (AA3)

### Énoncé

Le module `pickle` sérialise n'importe quel objet sans qu'on écrive de `to_dict`. Vous évaluez ce raccourci.

1. À la console, sérialisez votre catalogue avec `pickle.dumps`, puis reconstruisez-le avec `pickle.loads`. Vérifiez que les types et l'état sont préservés sans aucune méthode de sérialisation écrite par vous.
2. Ouvrez le fichier JSON de l'exercice 3 dans un éditeur de texte, puis tentez d'ouvrir un fichier `pickle` de la même manière. Notez la différence.
3. **Question AA3** (dans `reponses.txt`). Pour un catalogue destiné à être *relu plus tard, partagé entre outils et inspecté à la main*, quel format retenez-vous ? Justifiez par au moins trois arguments distincts portant sur : la lisibilité du format, le couplage à la définition des classes, et la sécurité au chargement d'un fichier d'origine non fiable.

- ✓ L'aller-retour `pickle` est démontré et la différence d'inspectabilité est constatée.
- ✓ Le format retenu pour un catalogue durable est JSON, justifié par trois arguments distincts.
- ✓ Les trois axes (lisibilité, couplage aux classes, sécurité) sont abordés.

## 6. Exercice 6 — Diagnostic d'un chargement (AA2+AA3)

### Énoncé

Vous recevez `chargement_buggy.py`. Il contient un catalogue JSON hétérogène et une fonction `charger_catalogue` censée le reconstruire.

1. Exécutez le fichier. Décrivez précisément le symptôme observé, *sans encore l'expliquer*.
2. Identifiez la cause. Nommez ce que la fonction `charger_catalogue` fait de la donnée présente dans le fichier mais ignorée à la relecture.
3. **Le test comme garde-fou (AA2).** Écrivez un test `unittest` qui, à *partir de la spécification* (« un livre numérique rechargé doit rester un livre numérique »), **échoue** sur la fonction buggy. Le test vérifie le type exact restauré — on utilise `assertIs(type(livre), LivreNumerique)` et non `assertIsInstance`, qui serait satisfait par le parent.
4. Proposez la correction de `charger_catalogue`, et vérifiez qu'elle fait passer votre test au vert.

Les valeurs que votre test affirme se déduisent de la **spécification** (le type qui aurait dû être restauré), pas de l'exécution du code buggy. Un test calé sur la sortie observée d'un code défectueux ne détecte jamais le défaut.

- ✓ Le symptôme est décrit précisément (types concrets perdus, champs propres absents après rechargement).
- ✓ La cause est nommée : le champ `"type"` n'est pas exploité, tout est reconstruit en `Livre`.
- ✓ Le test échoue sur la version buggy et utilise `assertIs` sur le type exact.
- ✓ La correction proposée fait passer le test au vert.

### Clôture

En fin de séance, votre fork doit contenir :

- le squelette `livre_s18_squelette.py` complété ;
- le squelette `catalogue_persistence_squelette.py` complété ;
- le test de l'exercice 6 et la correction de `charger_catalogue` ;
- `reponses.txt` renseigné (questions AA3 des exercices 2, 4 et 5, et diagnostic de l'exercice 6).

Ouvrez la Pull Request : elle est votre point de rendu.

La prochaine étape ouvre l'Axe 3 graphique avec PySide6 : la **programmation événementielle**, où une fenêtre relie des **signaux** à des méthodes par `.connect()` . Le catalogue persistant que vous venez de construire est exactement ce qu'une interface chargera au démarrage et sauvegardera à la fermeture.